

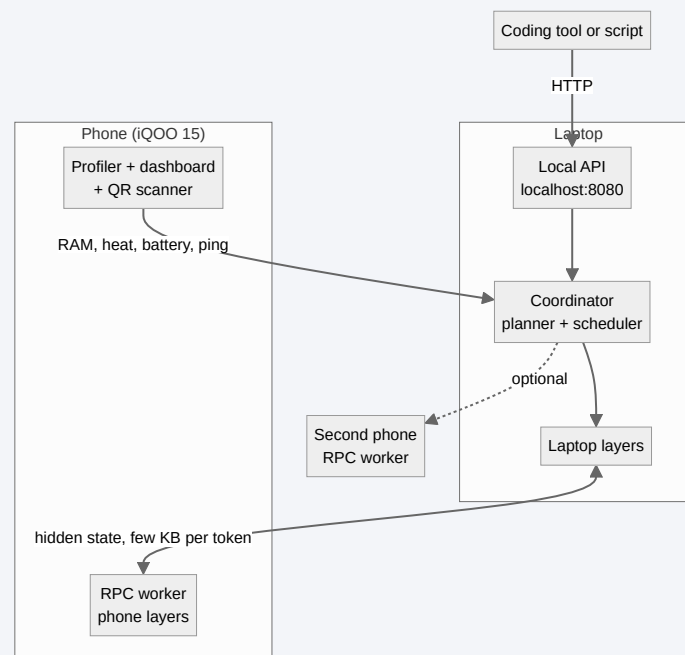
MeshAI Architecture

Team notes for our own understanding. Not part of the submission. 22 September 2026.

01 Big picture

Two apps on a private link: a laptop app that plans and serves, and a phone app that holds and runs part of the model.

llama.cpp does the maths on both sides. We build everything around it. The coding tool only ever sees one local address and never knows how many devices are behind it.



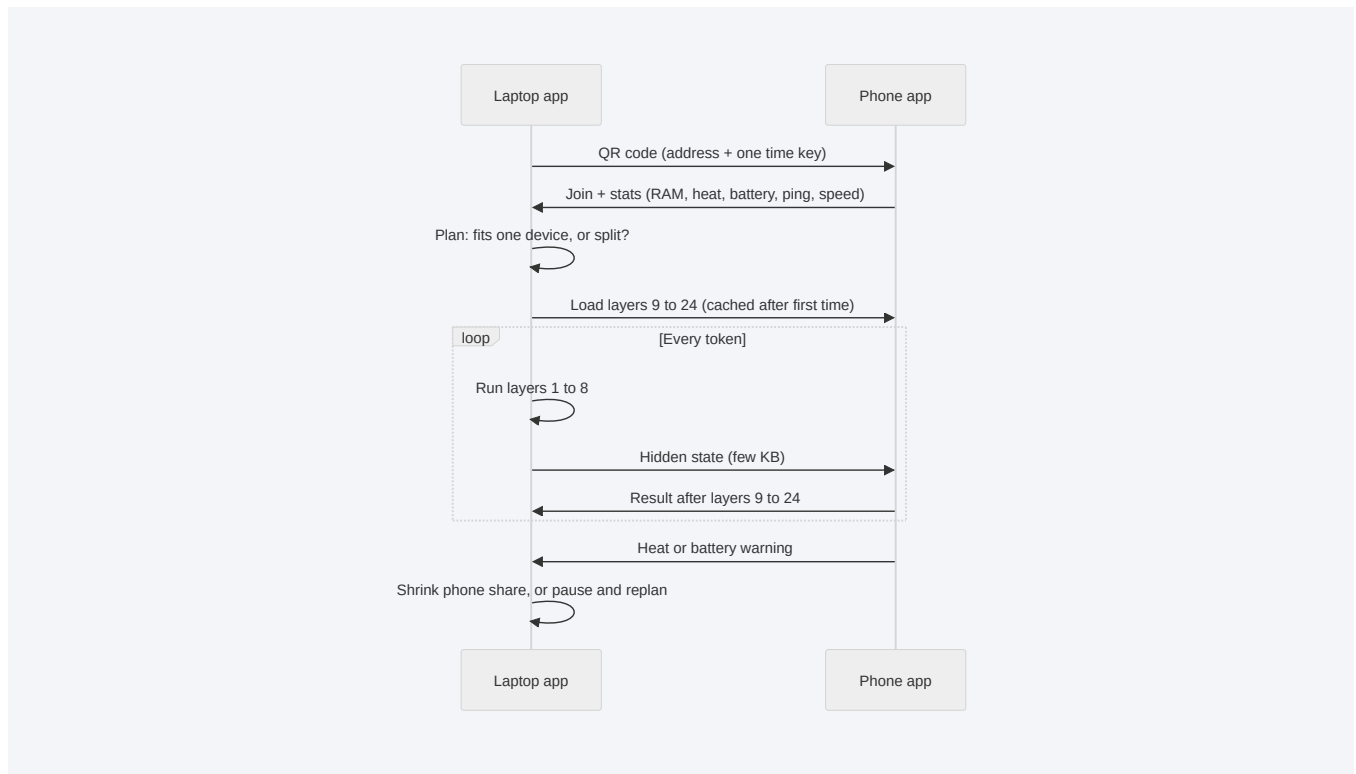
02 Components

Eight parts: five on the laptop, three inside the phone app.

Part	Runs on	What it does	Built by
Local API	Laptop	One OpenAI style address (/v1/chat/completions) that tools call.	llama.cpp server
Coordinator	Laptop	Keeps the list of paired phones and their latest stats.	Us
Model planner	Laptop	Reads the GGUF file: size of each layer, memory for context, headroom.	Us
Scheduler	Laptop	Decides which device gets which layers, or whether to split at all.	Us
Job queue	Laptop	Holds long tasks (write tests, review a branch) and runs them one by one.	Us
RPC worker	Phone	Holds its layers in RAM and computes them when asked. Started inside the app with <code>ggml_backend_rpc_start_server()</code> , so no terminal.	llama.cpp, wrapped by us
Profiler	Phone	Reads free RAM, temperature, battery and ping. Runs a quick speed test on join.	Us
Dashboard and QR scanner	Phone	The screen we watch: layers held, tokens per second, heat, battery. Must stay in the foreground or Android may kill the worker.	Us

03 Step by step

A session goes through six stages, from pairing to recovery.

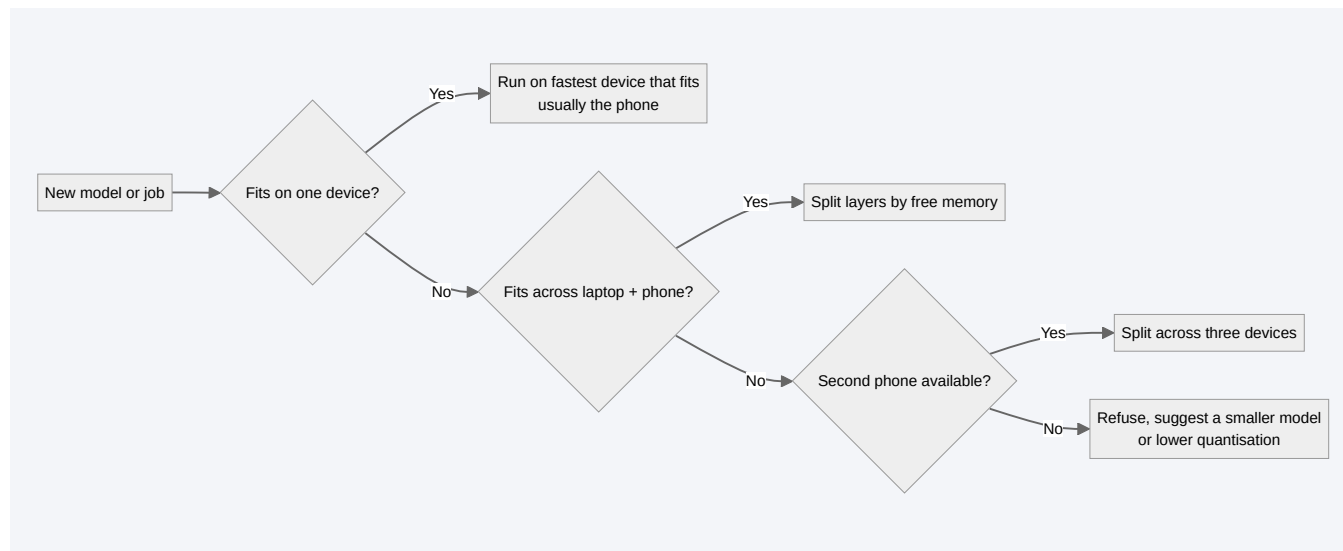


1. **Pair.** The laptop shows a QR code with its address and a one time key. The phone scans it and connects.
2. **Profile.** The phone reports its numbers, for example 9.8 GB free, 41°C, 80% battery, 4 ms ping.
3. **Plan.** The scheduler compares model size against free memory (rules in section 04).
4. **Load.** Each device loads only its own layers. The phone keeps its part cached. Office Kit file transfer can put the model file on the phone in advance.
5. **Serve.** For each token the laptop runs its layers, sends a small hidden state to the phone, and the phone runs its layers and sends the result back.
6. **Watch and recover.** If the phone heats up or the battery drops, layers move back or the job pauses. If a phone disconnects, stop cleanly, replan with the devices left, and restart the job.

Layer numbers are only an example. The real split comes from measured free memory.

04 Scheduler rules

Use the fewest devices that can hold the model. Add a device only for its memory, and drop it if it slows things down.



Separately, if the work is many independent jobs, each device gets its own job with a model that fits it (stretch goal).

Signal	Rule
Free memory	Hard limit. Keep 1 to 2 GB headroom on the phone so Android does not kill the app.
Backend speed (CPU, GPU, NPU)	Use the fastest one that gives correct output. CPU is the default until measured.
Ping	High or jumpy ping slows every token. Prefer our own hotspot.
Temperature	Shrink the phone's share or pause before it throttles hard.
Battery	Prefer a charging phone. Stop below a set minimum.
Repeated failures	Drop that device from the plan.

The headroom figure is our starting guess. We tune it once we see how the iQOO 15 behaves.

05 Network and memory

The model never moves over Wi-Fi during use. Only a small hidden state does, once per token, so ping matters far more than bandwidth.

What moves	Size	When
Model layers to the phone	about 8 GB	Once. Cached after that, or preloaded with Office Kit.
Long prompt, per device boundary	a few MB to tens of MB	Once per request, while reading the prompt.
Hidden state per token	a few KB	Every token, there and back.

Why ping matters. Every token waits for one round trip. At our target of 3 tokens per second (about 330 ms per token), a 5 ms hotspot ping costs little. A busy venue network with spikes of 250 ms would nearly double the time per token.

Worked memory example, gpt-oss-20b (12.1 GB).

Device	Free	Can hold, after headroom
Laptop, 8 GB	about 4 GB	about 3.5 GB
Phone, 16 GB	about 10 GB	about 8.5 GB
Total		about 12 GB, against a 12.1 GB model

Honest reading: one laptop plus one phone is just enough, with no room to spare. We must keep context short, close other apps, or add the second phone. If the loaner phones are the 12 GB version, one phone will not be enough and the second phone becomes part of the core demo.

06 Build, reuse, open choices

We reuse	We build
llama.cpp inference, RPC, GGUF loading, OpenAI style server	Phone app: worker wrapper, profiler, dashboard, QR pairing
Open models: gpt-oss-20b, Qwen3-Coder-30B-A3B	Coordinator, model planner, scheduler
Office Kit for file transfer and mirroring	Pairing flow, job queue, the "do not split if it fits" logic

Open choice	Option A	Option B
Phone app	Kotlin with Jetpack Compose	Flutter with a native plugin
Laptop coordinator	Python (FastAPI)	Go or Node.js
Phone compute	CPU	Adreno GPU (OpenCL), Hexagon NPU (experimental)
Link	Our own hotspot	USB tethering

Check first on real hardware, in this order:

1. How much RAM is actually free on the loaner phone (12 GB or 16 GB version).
2. Does the RPC worker on Android use the GPU, or fall back to CPU (llama.cpp issue #11957)?
3. Tokens per second on the phone alone, CPU against GPU, with a small model.
4. Ping over our hotspot, and over USB tethering.
5. Does gpt-oss-20b load and run split across laptop and phone?
6. Phone temperature and speed after 10 minutes of steady work.